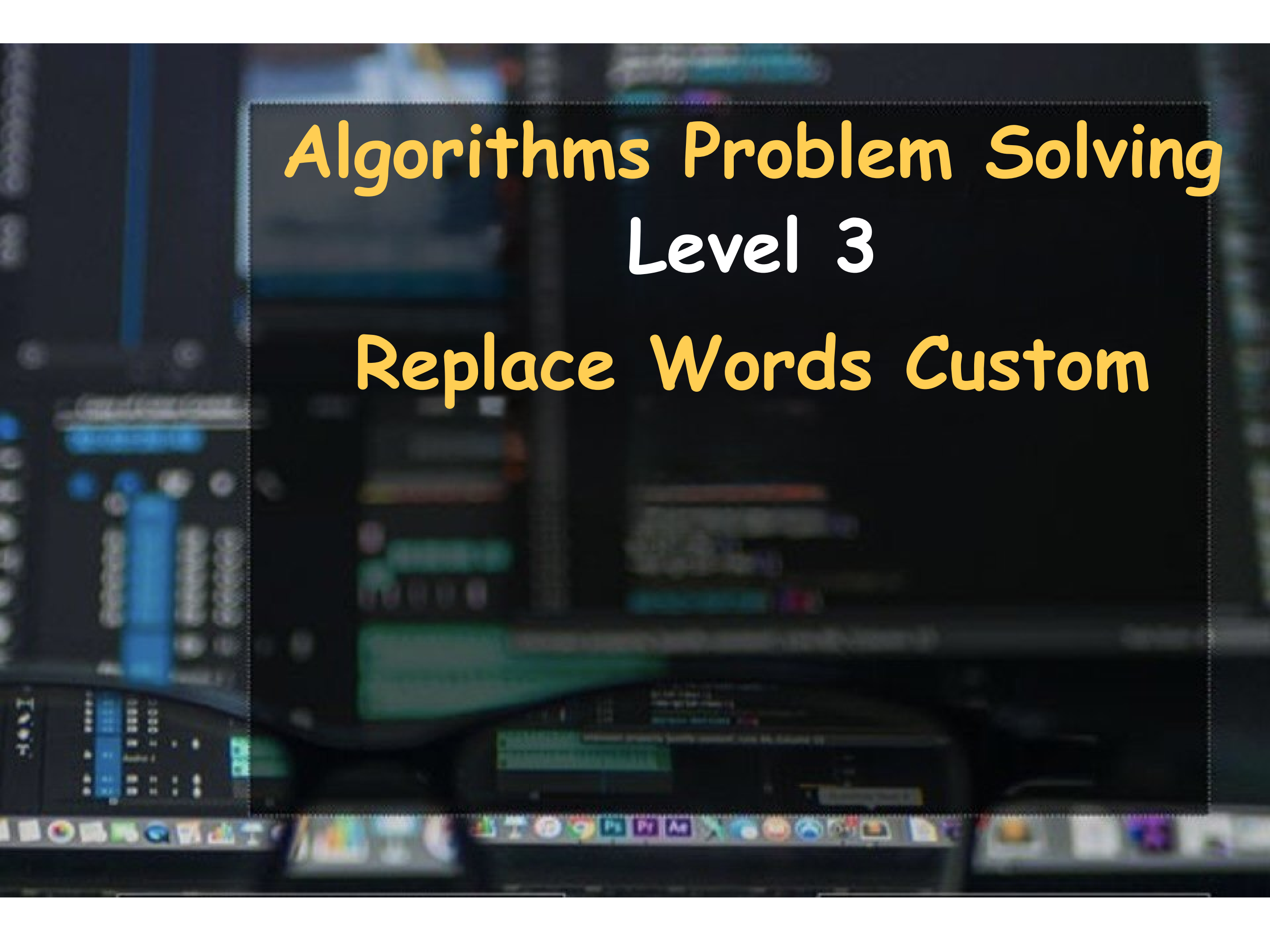




Algorithms Problem Solving

Level 3

Replace Words Custom



Algorithms Problem Solving

Level 3

Replace Words Custom

Definition

Token-Based Custom Word Replacement

 Program replaces target words by splitting a sentence into tokens and modifying matching vector elements.

 This approach provides greater control over whole-word and case-sensitive matching.

Explanation

Sentence Tokenization

-  The sentence is passed to ``SplitString()``.
-  Each word becomes an independent vector element.

Token Comparison

-  Each token is compared with the target word.
-  Case-sensitive or case-insensitive behavior can be selected by normalizing the strings before comparison.

Token Replacement

-  When a matching token is found, it is replaced directly inside the vector.
-  The replacement text can preserve its own desired casing.

Sentence Reconstruction

-  After all matching tokens are modified, ``JoinString()`` combines the vector back into a complete sentence.

Procedures

1. Read the sentence, target word, and replacement word.
2. Split the sentence into tokens.
3. Traverse the token vector.
4. Compare each token with the target.
5. Replace matching tokens.
6. Join the modified tokens.
7. Display the final sentence.

💡 Example

String:

Welcome to Sweden

Target:

Sweden

Replace:

Syria

➡ Result

Welcome to Syria

Note

- ◆ Token-based replacement avoids many accidental substring matches.
- ◆ Case-insensitive comparison can be performed using `ToLowerAllString()`.
- ◆ The replacement word itself does not need to be converted to lowercase when only comparison normalization is required.

⊘ Warning

- ◆ Punctuation attached to a token can prevent an exact match, such as `"Jordan,"` versus `"Jordan"`.
- ◆ Splitting only on spaces does not fully understand punctuation or advanced language word boundaries.
- ◆ Rejoining must use the correct delimiter to reconstruct the sentence properly.



Comparison

▨ Custom Token Replacement

- ✓ Compares complete tokens.
- ✓ Can support case-sensitive and case-insensitive matching.
- ✓ Provides more control over replacement behavior.

▤ `find()` / `replace()`

- ✓ Searches substrings directly.
- ✓ Is simpler for general text replacement.
- ✓ Can accidentally replace partial words.

✦✦ Summary

- ✓ Splits sentences into individual word tokens.
- ✓ Compares tokens using custom matching rules.
- ✓ Replaces matching vector elements.
- ✓ Reconstructs the sentence using `JoinString()`.

Thank you 😊